# TerraMind on SmallMinesDS: Experiment Report

Oct 21, 2026 · Asım Emre Arslan

## Summary

With 50 epochs of training and a cosine learning-rate schedule from 5e-3 to 1e-5, the frozen pretrained TerraMind v1 tiny encoder beats the same architecture trained from scratch at every budget from N = 10 upward, in all five seeds, and the margin grows with data: mine-class IoU 0.683 vs 0.605 at N = 250 without augmentation, 0.707 vs 0.584 with it. D4 augmentation adds a consistent +0.02 to +0.04 IoU for the frozen encoder and nothing reliable for the trainable one.

- **Every budget learns, including a single image**: IoU 0.36–0.40 at N = 1 in all four conditions.
- **Data efficiency.** The frozen encoder with augmentation reaches with 10 images (IoU 0.591) what the trainable random encoder reaches with 250 (0.584–0.605); with 20 images (0.634) it is beyond anything the random encoder achieves at any budget.
- **Augmentation** helps the frozen encoder by +0.016 to +0.038 IoU, 5/5 seeds at every N ⩾ 3. For the trainable random encoder the effect is −0.021 to +0.046, changes sign with N and improves 2–5 of 5 seeds.
- **Seed variance** is small for the frozen encoder (std ⩽ 0.013 at N ⩾ 20, 0.002 at N = 250 with augmentation) and 2–3× larger for the random encoder (up to 0.032).
- **Fully reproducible.** 160 runs in 68 minutes on one RTX 3080 Laptop GPU, bit-for-bit repeatable from the public repository.

## Research question and design

The plan asks how training-set size and geometric augmentation affect binary mine segmentation when a frozen pretrained TerraMind v1 tiny encoder is compared with the same encoder architecture trained from scratch. The design is a 2 × 2 of encoder strategy and augmentation, crossed with eight training budgets and five seeds: 160 runs.

| Condition | Encoder initialisation | Encoder training | Augmentation |
|---|---|---|---|
| A | TerraMind v1 tiny, pretrained | frozen | none |
| B | TerraMind v1 tiny, pretrained | frozen | random D4 symmetry |

| Condition | Encoder initialisation | Encoder training | Augmentation |
|---|---|---|---|
| C | random | trainable | none |
| D | random | trainable | random D4 symmetry |

- Budgets N ∈ {1, 3, 5, 10, 20, 50, 100, 250} unique training images; subsets nested within a seed and identical across the four conditions.
- Protocol: exactly 50 epochs, evaluate the final weights; AdamW with a per-step cosine learning-rate schedule from 5e-3 (first update) to 1e-5 (last update), weight decay 0.01, batch size min(8, N); unweighted 2-class pixel cross-entropy; reshuffle every epoch, final partial batch kept; no early stopping, validation set or checkpoint selection. Because the epoch count is fixed, optimiser steps grow with N: 50 for N ⩽ 5, 100 at N = 10, 150 at N = 20, 350 at N = 50, 650 at N = 100, 1600 at N = 250.
- Augmentation: one of the eight square symmetries (rotations by 0/90/180/270°, each with or without a horizontal reflection), drawn uniformly per image presentation and applied identically to S1, S2 and the mask; test inputs never augmented; N unchanged.
- Metrics: mine-class IoU, F1, precision and recall from argmax predictions and confusion counts pooled over the fixed 644-image test set; zero-denominator metrics reported as NA; curves of mean ± std over seeds against N; paired augmentation gains within each encoder strategy.
- Caveat from the plan itself: the encoder comparison changes initialisation and trainability at once.

## Data and preprocessing

SmallMinesDS 2022 holds 2,135 patches of 128 × 128 px over five districts of south-western Ghana, split by the dataset authors into 1,491 training and 644 test patches. Masks are binary (1 = mine); 7.6 % of all pixels and 7.8 % of test pixels are mine, and 511 training patches (34 %) contain no mine pixels at all. The pinned dataset revision is downloaded and checksum-verified by a script; every CSV row was matched to its image and mask file, and the CSV mine percentages agree with the masks to 0.005 %.

Inputs are the ten Sentinel-2 L2A bands B2, B3, B4, B5, B6, B7, B8, B8A, B11, B12 and the two Sentinel-1 RTC bands VV, VH; the DEM band is dropped as the plan requires. Sentinel-1 is stored linear in the GeoTIFFs and converted to dB (10·log10) because TerraMind was pretrained on dB values. Both modalities are standardised with TerraMind's own pretraining mean and standard deviation, in every condition, so the four conditions see identical inputs.

The plan left several choices open; each was decided before any run and is recorded in the repository README.

| Open point in the plan | Decision | Reason |
|---|---|---|
| Input size | Native 128 × 128, no resizing | Keeps the data as published; TerraMind's fixed sin-cos position table for 128 px is exactly the top-left 8 × 8 crop of its 224 px table, so pretrained weights load unchanged |
| Normalisation | TerraMind pretraining statistics for all conditions | Only the encoder should differ between conditions |
| Sentinel-1 flavour | S1RTC modality, values in dB | Dataset radar is RTC and speckle-filtered; TerraMind expects dB |
| S1 + S2 fusion | TerraTorch: joint token sequence in the encoder, per-layer outputs averaged (merge_method = mean) | Library default; keeps the decoder identical and small |
| Shallow decoder | 4-block UNet decoder on a learned 4-level pyramid (details below) | Chosen by the project owner; identical in every condition |
| Subsets at small N | First 10 images of each seed's ordering drawn from the 140 training patches with 25–75 % mine pixels; the rest in random order | Avoids N = 1 subsets with no mine pixels (34 % of patches); keeps budgets nested; N ⩽ 10 are all such images and larger budgets contain at least those 10 |
| Seed | One seed fixes subset, decoder initialisation and per-epoch batch order; augmentation draws from a separate seeded stream | Makes the four conditions of a seed a paired comparison |

The subset rule departs from the plan's pure random ordering and makes small-N training sets far more mine-heavy (25–75 % of pixels) than the test set (7.8 %); precision and recall at N ⩽ 20 should be read with that in mind. The exact patch ids of every (seed, N) subset are stored in `results/main/subsets.json`.

## Model

The model is built with TerraTorch 1.2.13 and is identical in every condition; only the encoder's weights and trainability change.

- **Encoder**: `terramind_v1_tiny` (12 layers, width 192, patch 16). S2L2A (10 bands) and S1RTC (2 bands) are embedded separately, processed as one joint token sequence, and the two modalities' outputs are averaged per layer. At 128 px each modality yields 8 × 8 tokens. In the pretrained conditions the weights come from TerraMind-1.0-tiny; all 152 weight tensors load unchanged at 128 px, and the 10-band Sentinel-2 patch embedding is the matching slice of the pretrained 12-band one.
- **Neck**: outputs of encoder layers 2, 5, 8 and 11 are reshaped to 8 × 8 maps and turned into a 4-level pyramid (32, 16, 8, 4 px) by TerraTorch's learned interpolation neck.
- **Decoder and head**: a 4-block UNet decoder (segmentation_models_pytorch) with 128/64/32/16 channels up to 64 × 64, a 1 × 1 convolution to two classes, bilinear × 2 to 128 × 128. Normalisation layers are BatchNorm2d (library default) in the main experiment; supplementary experiment A replaces them with GroupNorm.

| Part | Parameters | Trainable in frozen-encoder conditions |
|---|---|---|
| Encoder (TerraMind v1 tiny) | 5.92 M | no |
| Neck + decoder + head | 0.97 M | yes |
| Total | 6.90 M | 0.97 M of 6.90 M |

The decoder side is re-initialised from a seed-derived generator after the encoder is built, so for a given seed its initial weights are the same in all four conditions.

## Training protocol and reproducibility

Every run follows the same protocol: 50 epochs; AdamW (weight decay 0.01) with a cosine learning-rate schedule applied per optimiser step, 5e-3 at the first update and 1e-5 at the last; batch size min(8, N); unweighted two-class pixel cross-entropy; a fresh shuffle each epoch with the final partial batch kept; evaluation of the final weights on the full test set; no early stopping or model selection. The number of optimiser steps grows with N: 50 for N ⩽ 5, 100 at N = 10, 150 at N = 20, 350 at N = 50, 650 at N = 100 and 1600 at N = 250. No run diverged at the 5e-3 starting rate.

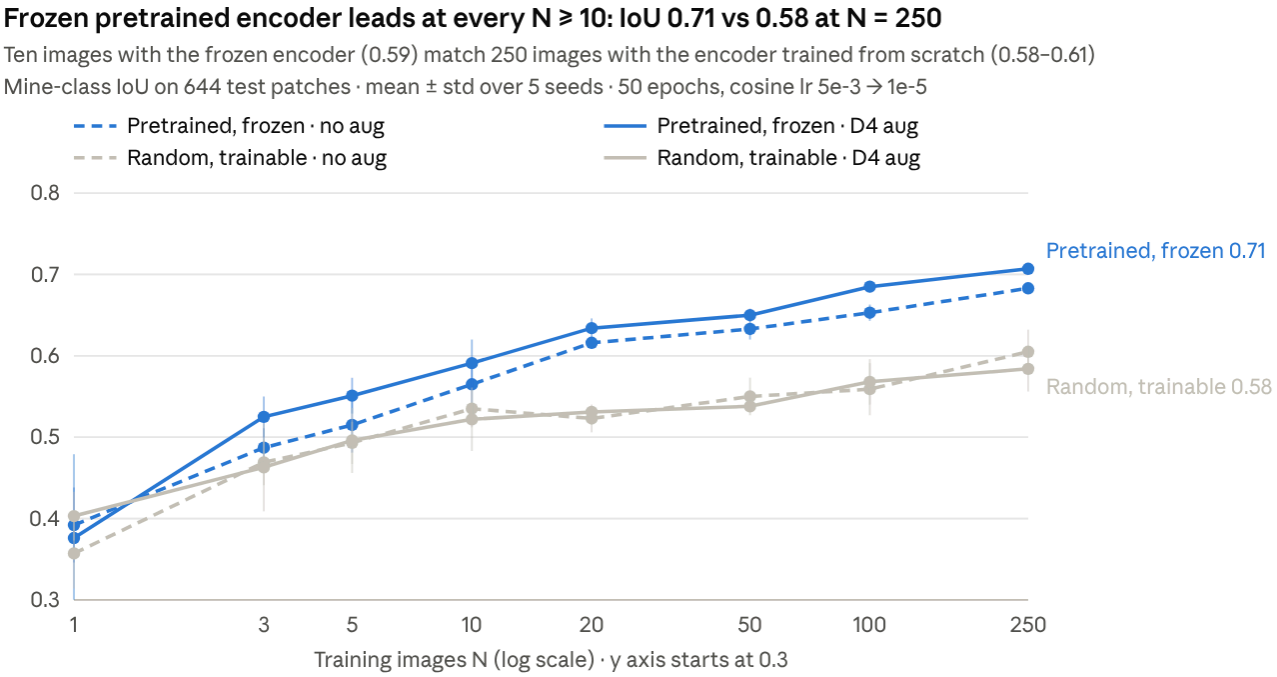
Within a seed the four conditions are paired: they train on the same images, start from the same decoder weights and see the same batch order. Augmentation draws come from their own seeded generator, so switching augmentation on does not change which images share a batch. The random-init encoder is also initialised from the seed.

Runs are bit-for-bit reproducible on the same machine and software stack: cuDNN deterministic mode, `CUBLAS_WORKSPACE_CONFIG=:4096:8`, `torch.use_deterministic_algorithms(True)`, the math attention kernel instead of the

non-deterministic flash kernels, and a cross-entropy written without the non-deterministic `nll_loss2d` CUDA kernel. Each configuration was repeated three times in a test and produced identical confusion counts and loss values; the refactored training loop used for the supplementary runs reproduces the stored main-experiment rows exactly.

## Results of the main experiment

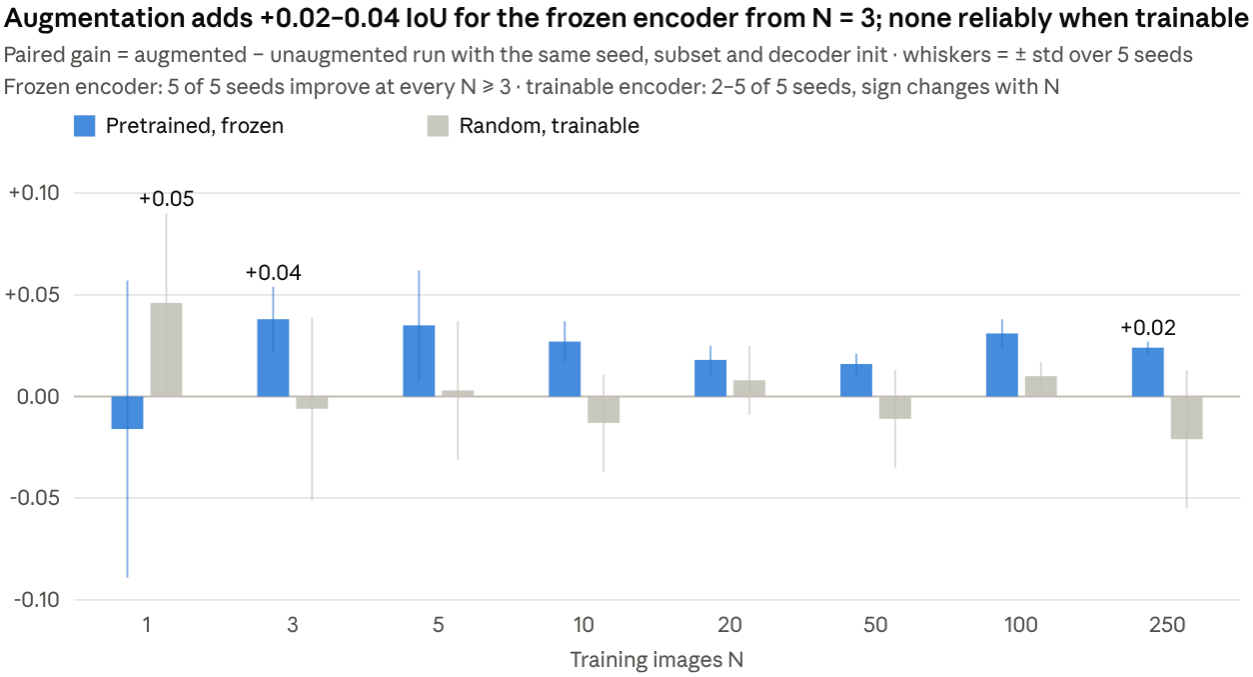
The frozen pretrained encoder is the better strategy at every budget from N = 10, augmentation adds a small, consistent gain for it, and even a single training image yields a usable model. All 160 runs completed in 68 minutes with no divergence and no NA metric.




results/main/summary.csv · 160 runs: 8 budgets × 4 conditions × 5 seeds

- **Every budget learns.** At N = 1 all four conditions reach IoU 0.36–0.40 from a single training image.
- **The frozen pretrained encoder wins at every N ≥ 10 in 5/5 seeds.** The margin grows with N: +0.02–0.04 IoU at N ≤ 10, +0.08–0.12 from N = 20 on (0.683 vs 0.605 without augmentation at N = 250, 0.707 vs 0.584 with it). F1 at N = 250: 0.828 vs 0.737.
- **Data efficiency.** The frozen encoder with augmentation reaches with 10 images (IoU 0.591) what the trainable random encoder reaches with 250 (0.584–0.605); with 20 images (0.634) it is beyond anything the random encoder achieves at any budget.
- **Precision and recall.** The frozen encoder's advantage is mostly recall (0.77–0.81 vs 0.65–0.71 at N ≥ 20); its precision is also higher from N = 20 (0.76–0.85 vs 0.73–0.80). At N ≤ 10 precision and recall reflect the mine-heavy subsets in every condition.

- **Seed variance** is small for the frozen encoder (std ⩽ 0.013 at N ⩾ 20, 0.002 at N = 250 with augmentation) and 2–3× larger for the random encoder (up to 0.032), so differences of ⩾ 0.02 between the encoders are interpretable while most augmentation effects on the random encoder are not.




results/main/aug_gain.csv · paired differences, 5 seeds per cell

For the frozen encoder the paired gain is positive in every seed at every N ⩾ 3 (+0.016 to +0.038 IoU); at N = 1 it is −0.016 ± 0.073. For the trainable random encoder the gain ranges from −0.021 to +0.046, changes sign with N and improves 2–5 of 5 seeds.

Full tables for all four metrics are in `results/main/summary.md`; the per-run rows, including confusion counts and per-epoch training loss, in `results/main/runs.csv`.

## Discussion

With a sufficient optimisation budget the picture is simple: on this dataset, frozen pretrained features generalise better than features learned from scratch at every budget that matters, and augmentation is worth having with the frozen encoder.

- **Data efficiency.** The frozen encoder keeps improving with labelled data (IoU 0.616 → 0.683 from N = 20 to 250 without augmentation) while the trainable encoder gains less over the same range (0.523 → 0.605). With 5.9 M encoder parameters and at most 250 images, learning the representation from scratch is the bottleneck; reading pretrained features through a 1 M-parameter decoder is not.

- **Augmentation.** With a frozen encoder only the decoder learns, and the eight orientations of each image act as a steady regulariser: +0.02 to +0.04 IoU at every budget from N = 3, in every seed. With a trainable encoder the extra orientation variety has to be absorbed by the encoder itself; within 50 epochs that neither helps nor hurts consistently.
- **Encoders.** Initialisation and trainability change together, so the result reads “frozen pretrained encoder + shallow decoder beats the same architecture trained from scratch”; it does not say how much fine-tuning the pretrained encoder would add or cost.
- **Budgets.** Because the epoch count is fixed, optimiser steps grow with N (50 to 1600), so the curve against N mixes more data with more optimisation. The comparison between conditions at a given N is unaffected: all four receive the same steps, images and batch order. At N ⩽ 5 the 50 steps with a 5e-3 start are enough for a non-trivial model (IoU 0.36–0.40).
- **Variance.** The frozen encoder's seed spread (std ⩽ 0.013 at N ⩾ 20) makes differences of 0.02 or more interpretable; the random encoder's 0.02–0.03 spread means its augmentation effects (|gain| ⩽ 0.05) are not.

## Limitations and deviations from the plan

Everything below is stated in the repository README as well; none of it changes the ordering of conditions at N ⩾ 10, but each affects how far the numbers generalise.

- **Confounded encoder comparison.** Initialisation and trainability differ at once, as the plan itself notes. A fifth condition, pretrained and trainable, is needed to separate them and was not run.
- **Step budget scales with N.** With a fixed epoch count the budgets receive 50 to 1600 optimiser steps, so they are not equal in optimisation; an equal-step variant was not run (about 4.5 h at 1600 steps per run).
- **Subset rule.** The first 10 images of every seed are drawn from patches with 25–75 % mine pixels, not at random. Small-N training sets are therefore far more mine-heavy than the 7.8 % test base rate, which shapes precision and recall at N ⩽ 10. The four conditions share the same subsets, so comparisons between them are unaffected.
- **Decoder choices are ours.** The plan says only “shallow decoder”; the UNet depth, channel widths and BatchNorm layers were chosen here and were not varied.
- **Resolution.** Images are used at their native 128 px instead of TerraMind's 224 px pretraining resolution. The positional table crop is exact, but the encoder sees 64 tokens per modality rather than 196, and feature quality at this size was not validated separately.

- **Five seeds.** Differences smaller than one standard deviation (the encoder difference at N = 1, most augmentation effects on the random encoder) should not be interpreted.
- **One dataset, one year, one region.** Results describe SmallMinesDS 2022 in south-western Ghana with its published split. There is no validation set by design; the test set was used once per run and never for any selection.
- **Reproducibility** is bit-exact on the machine and software stack recorded in `results/main/environment.json`; other GPUs or driver versions may differ in the last digits.

## Recommendations and next steps

1. **Add a pretrained, trainable condition** (fine-tuning) with the same seeds, subsets and protocol: about 35 minutes for 80 runs with the existing scripts. It separates initialisation from trainability, the one question this grid cannot answer.
2. **If small-N numbers will be quoted on their own**, run an equal-step variant (`--steps 1600`, about 4.5 h) or adopt a step floor, so that N ⩽ 10 and N = 250 receive the same optimisation.
3. **Run a purely random subset variant** as a sensitivity check on the mine-fraction rule; expect higher seed variance and, in roughly a third of seeds, N = 1 subsets without any mine pixels.
4. **Check the resolution choice** by repeating the frozen-encoder conditions at 224 px (bilinear upsampling); if IoU moves by more than the seed spread, the 128 px numbers should be labelled as such.
5. Optional: the 2016 split of the same dataset allows a temporal transfer test of the N = 250 models at no training cost.

## Reproducibility appendix

Code, results and figures: github.com/asimarslan11/SmallMinesDS-Experiment-Project (results under `results/main/`, as of 21 October 2026). The dataset is not in the repository; the download script fetches revision `0cb168b8` of SmallMinesDS from Hugging Face and verifies its checksum, and TerraMind-1.0-tiny weights are downloaded by TerraTorch on first use.

```
python -m venv .venv && .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts\check_gpu.py
python scripts\download_data.py
python scripts\build_cache.py
python scripts\run_experiments.py   # 160 runs, ~70 min on an RTX 3080
Laptop; resumable
python scripts\analyze.py           # tables and figures -> results/main/
```

| Item | Value |
|---|---|
| Hardware | NVIDIA GeForce RTX 3080 Laptop GPU, 8 GB; Windows 11 |
| Software | Python 3.12.0, PyTorch 2.11.0+cu128, CUDA 12.8, cuDNN 9.19, TerraTorch 1.2.13, segmentation_models_pytorch 0.5.0, rasterio 1.5.2 |
| Compute | 160 runs, 4,090 s (68 min) in total; 2–130 s per run depending on N |
| Per-run record | seed, N, condition, confusion counts, IoU/F1/precision/recall, steps, first and last learning rate, per-epoch training loss, subset mine fraction, wall time, parameter counts |
| Subsets | `results/main/subsets.json`: patch ids of every (seed, N) training subset |
| Summary files | `results/main/summary.md`, `summary.csv`, `aug_gain.csv`, `figures/` |
| Determinism | cuDNN deterministic, `CUBLAS_WORKSPACE_CONFIG=:4096:8`, `torch.use_deterministic_algorithms(True)`, math attention kernel, deterministic cross-entropy; repeated runs identical in confusion counts and loss |

Sources: SmallMinesDS dataset card, TerraMind 1.0 tiny model card, TerraTorch, and the project plan document *TerraMind v1 Tiny on SmallMinesDS — Experiment plan* (SmallMinesDS_TerraMind_Project_v1.docx).